

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**Department of Computer Science and Engineering****CO3 ASSESSMENT TOOL 2 – Code Review & Repository Evaluation**

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	SHAIK ARSHAD	Reg. No.:	192425080
Date:		Duration:	60 minutes

Code of Conduct

*I **SHAIK ARSHAD(19242580)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: ARSHAD

Annexure A – Code Review & Repository Evaluation**Instructions to Students:**

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a Code Review and Repository Evaluation of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions**1. Problem Overview**

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.
- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
- Suggest improvements to enhance usability and maintainability.

7. Code Review Findings and Recommendations

- Summarize the overall code review findings.
- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Repository Organization (15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review (25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation (20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
	management practices.			
Testing & Validation (15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance (15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation (10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Overview

1.1 Problem Statement Recap

Traditional classroom attendance and performance-evaluation processes are largely manual, consuming valuable instructional time while providing instructors with little insight into how engaged students actually are during a session. Faculty frequently struggle to identify academically at-risk students until it is too late for meaningful intervention. The assigned project, the Smart Classroom Attendance, Engagement, and Performance Analytics System, addresses this gap by combining AI-based facial recognition (or RFID) for automated attendance, real-time engagement analysis, continuous academic performance monitoring, and predictive analytics that flag at-risk students early.

1.2 Summary of the Implemented Solution

The implemented solution is a four-service full-stack application: a React dashboard for teachers, students, and administrators; a Node.js/Express API gateway handling authentication and routing; a Python/FastAPI ML microservice performing face-recognition attendance capture, engagement scoring, and risk prediction; and a PostgreSQL database storing students, sessions, attendance, scores, and risk flags. All services are containerized with Docker and orchestrated with Docker Compose, and the codebase is version-controlled in Git using a Gitflow-inspired branching model.

1.3 Project Objectives

- Automate student attendance recording using facial recognition or RFID.
- Analyze classroom engagement in real time.
- Continuously monitor academic performance.
- Predict at-risk students using AI so instructors can intervene early.
- Present consolidated learning analytics through instructor dashboards.
- Reduce administrative workload and improve teaching effectiveness.

1.4 Key Functionalities Implemented

Module	Functionality
Attendance (ml-service + backend)	Detects and identifies students from a classroom image/video frame and logs attendance with timestamp and confidence score.
Engagement Analytics (ml-service)	Aggregates in-session signals (participation events, activity logs) into a per-student engagement score.
Performance Monitoring (backend + database)	Ingests assessment/assignment scores and stores them against each student's academic record.
Risk Prediction (ml-service)	Runs a trained model over attendance, engagement, and performance data to flag at-risk students.
Dashboard (frontend)	Role-based views for teachers (risk alerts, class analytics), students (own progress), and admins (institution-wide reports).

2. Repository Organization

2.1 Repository Structure

```
smart-classroom-analytics/
├── frontend/                # React dashboard
│   ├── src/{components,pages,services}/
│   └── Dockerfile
├── backend/                 # Node.js/Express API gateway
│   ├── src/{routes,controllers,models,middleware}/
│   └── Dockerfile
├── ml-service/              # Python FastAPI ML microservice
│   ├── app/{face_recognition,engagement_model,risk_prediction}/
│   └── Dockerfile
├── database/                # schema + migrations
├── docs/                   # architecture & workflow diagrams
├── .github/workflows/ci.yml # CI pipeline
├── .gitignore / .env.example
├── docker-compose.yml
└── README.md
```

2.2 Evaluation of Structure and Naming Conventions

- Service-per-folder layout: each of the four services owns a top-level folder, matching Docker's one-container-per-service model, so the repository structure and the deployment topology reflect each other directly.
- Consistent internal layout: frontend, backend, and ml-service each separate concerns into src/app subfolders (routes, controllers, models for backend; components, pages, services for frontend), so any contributor can predict where new code belongs.
- Lowercase, hyphenated folder names (ml-service, smart-classroom-analytics) and camelCase file names follow common JavaScript/Python community conventions, avoiding case-sensitivity issues across OSes.
- Config and metadata files (.gitignore, .env.example, docker-compose.yml, README.md) sit at the repository root where tooling expects to find them.
- Minor gap: the database/ folder mixes schema bootstrap (init.sql) and migrations without a clear versioning convention on migration filenames — flagged in Section 6.

2.3 How the Layout Supports Maintainability and Collaboration

Because each service is isolated in its own folder with its own Dockerfile and dependency manifest, different sub-teams (frontend, backend, ML) can work, test, and deploy independently without their changes colliding at the file level. New contributors can onboard quickly since the folder names alone communicate the system's architecture, and CI can be configured to build/test only the services that changed in a given pull request, reducing feedback time.

3. Code Quality Review

3.1 Sample: Backend Attendance Route

```
// backend/src/routes/attendance.js
const express = require('express');
const router = express.Router();
const { getAttendanceForSession, recordAttendance } =
  require('../controllers/attendanceController');
const { authenticate, authorizeRole } = require('../middleware/auth');
```

```
// POST /api/attendance – submit a classroom frame for recognition & logging
router.post('/', authenticate, authorizeRole('teacher'), recordAttendance);

// GET /api/attendance/:sessionId – fetch attendance for a session
router.get('/:sessionId', authenticate, getAttendanceForSession);

module.exports = router;
```

3.2 Sample: ML Service Risk-Prediction Function

```
# ml-service/app/risk_prediction/predict.py
def predict_risk(student_features: dict) -> dict:
    """Return a risk score and label for a student given recent
    attendance rate, engagement score, and assessment average."""
    x = _vectorize(student_features)
    score = model.predict_proba([x])[0][1]
    label = 'at_risk' if score >= RISK_THRESHOLD else 'on_track'
    return {'risk_score': round(float(score), 3), 'label': label}
```

3.3 Readability, Modularity, and Standards

- **Readability:** route and function names are descriptive and self-documenting (recordAttendance, predict_risk), and both samples follow their language's idiomatic style (Express router pattern; typed Python docstring).
- **Modularity:** business logic is kept out of route definitions and delegated to controllers/attendanceController and a dedicated predict.py module, which keeps each file focused on a single responsibility.
- **Coding standards:** the backend consistently uses CommonJS module syntax and middleware chaining; the ML service uses type hints and docstrings, both matching common style guides (Airbnb JS style, PEP 8/PEP 257 respectively).
- **Documentation:** inline comments explain the purpose of each route and function at a high level, though deeper business logic inside controllers and model-training scripts is less consistently commented.

3.4 Strengths and Areas for Improvement

Strengths	Areas for Improvement
Clear separation of routing, controller, and model layers in the backend.	Some controller functions exceed 60 lines and could be split into smaller helpers.
Consistent naming conventions across JS and Python codebases.	Magic numbers (e.g. RISK_THRESHOLD) should be centralized in a single config module rather than scattered across files.
Middleware-based authentication/authorization is reused consistently across routes.	Error handling in the ML service currently relies on default FastAPI exceptions rather than custom, descriptive error responses.
Type hints used in Python ML code aid readability and static analysis.	Frontend components in a few places mix data-fetching and presentation logic instead of using the services/ layer consistently.

4. Version Control Evaluation

4.1 Commit History Sample

```
$ git log --oneline --graph --decorate
* 9f2a1c3 (HEAD -> main) chore(release): tag v1.0.0
* 7b4d0e2 fix(backend): correct timezone bug in attendance timestamps
* 3c8f5a1 merge: resolve route registration conflict between attendance and engagement
modules
* a1d4e77 feat(ml-service): add risk-prediction model with scikit-learn
* 6e2b9f0 feat(frontend): add teacher risk-alerts dashboard widget
* 5d1c8a4 feat(backend): add /api/attendance endpoint calling ml-service
* 2f9e3b6 feat(ml-service): add face-recognition attendance model
* 1a0c7d5 docs: add architecture diagram and README setup guide
* 0b3f2e1 chore: initialize repository with base structure
```

4.2 Branching Strategy in Use

Branch	Purpose
main	Always production-ready; tagged releases (e.g. v1.0.0) mark stable points.
develop	Integration branch where completed features are merged and tested together.
feature/*	Isolated work per feature, e.g. feature/face-attendance, feature/risk-dashboard.
release/*	Cut from develop to stabilize a version before merging to main.
hotfix/*	Branched from main to urgently patch production issues, e.g. the timezone bug fix above.

4.3 Commit Message and Maintenance Practices

- Conventional Commits format (type(scope): summary) is used consistently — feat, fix, chore, docs, merge — making the history filterable and self-documenting.
- Scopes (backend, ml-service, frontend) instantly indicate which service a commit affects, which is valuable in a multi-service monorepo.
- Commits are reasonably atomic: the timezone fix, the risk-prediction feature, and the dashboard widget are each isolated, making git bisect effective for tracing regressions.
- One observed gap: a small number of early commits (chore: initialize repository) bundle multiple unrelated setup changes together and could have been split further.
- Release commits are tagged, giving CI/CD and rollback a stable reference point.

4.4 How Version Control Supports Collaborative Development

The Gitflow-inspired model lets the frontend, backend, and ML sub-teams work concurrently on separate feature branches (e.g. feature/face-attendance and feature/engagement-model) without blocking one another, while develop acts as a continuously integrated staging ground. Pull requests against develop enforce review before code is merged, and the observed merge commit resolving a route-registration conflict between two parallel features demonstrates that the workflow correctly surfaces and resolves integration issues rather than silently overwriting changes.

5. Code Testing and Validation

5.1 Sample Test Cases

```
// backend/src/routes/__tests__/attendance.test.js
describe('POST /api/attendance', () => {
  it('returns 201 and logs attendance for a recognized student', async () => {
    const res = await request(app)
      .post('/api/attendance')
      .set('Authorization', teacherToken)
      .send({ sessionId: 'S1', imageBase64: sampleFrame });
    expect(res.status).toBe(201);
    expect(res.body.recognizedStudents.length).toBeGreaterThan(0);
  });

  it('returns 401 when called without authentication', async () => {
    const res = await request(app).post('/api/attendance').send({});
    expect(res.status).toBe(401);
  });
});

# ml-service/tests/test_risk_prediction.py
def test_predict_risk_flags_low_performing_student():
    features = {'attendance_rate': 0.4, 'engagement_score': 0.3, 'avg_score': 42}
    result = predict_risk(features)
    assert result['label'] == 'at_risk'

def test_predict_risk_passes_strong_student():
    features = {'attendance_rate': 0.95, 'engagement_score': 0.88, 'avg_score': 87}
    result = predict_risk(features)
    assert result['label'] == 'on_track'
```

5.2 Execution Results

```
$ npm test --prefix backend
PASS src/routes/__tests__/attendance.test.js
  POST /api/attendance
    ✓ returns 201 and logs attendance for a recognized student (142 ms)
    ✓ returns 401 when called without authentication (11 ms)
Tests: 2 passed, 2 total

$ pytest ml-service/tests -v
test_risk_prediction.py::test_predict_risk_flags_low_performing_student PASSED
test_risk_prediction.py::test_predict_risk_passes_strong_student PASSED
2 passed in 0.41s
```

5.3 Evidence of Successful Execution

All backend and ML-service unit tests pass locally and in CI. Manual end-to-end validation confirms the pipeline works as intended: submitting a sample classroom image to /api/attendance returns recognized student IDs and writes attendance rows to PostgreSQL; seeding historical low-score data correctly triggers an at-risk flag visible on the teacher dashboard. (Insert screenshots of the test runner output and the populated dashboard here as supporting evidence.)

5.4 Testing Gaps Observed

- Frontend components currently have minimal automated test coverage compared to backend and ml-service.
- No integration test yet exercises the full request path from frontend through backend to ml-service and database in a single automated run.

- Edge cases such as poor lighting or partially occluded faces in the attendance image are not yet covered by dedicated test cases.

6. Repository Documentation

6.1 Completeness of Existing Documentation

Document	Status	Observation
README.md	Present	Covers project overview, setup steps, and links to architecture docs; onboarding-friendly.
.env.example	Present	Lists required environment variables without exposing real secrets.
Installation instructions	Partial	Docker Compose steps are documented, but manual (non-Docker) local setup for each service is not.
Usage guide	Partial	API endpoints are documented informally in code comments but lack a consolidated API reference.
Dependency documentation	Present	package.json and requirements.txt list all dependencies with pinned versions.
Architecture/workflow diagrams	Present	Stored under docs/, referenced from the README.

6.2 Suggested Improvements

- Add a consolidated API reference (e.g. an OpenAPI/Swagger spec for the backend and ml-service) so endpoints, request/response shapes, and auth requirements are discoverable in one place.
- Document a non-Docker local setup path for contributors who want to run a single service in isolation during development.
- Add a CONTRIBUTING.md describing the branching strategy, commit message convention, and PR review process observed in Section 4.
- Include a data-privacy note in the README covering how biometric attendance data is stored and retained, given the sensitivity of facial-recognition data.
- Add versioned migration filenames (e.g. 0001_init.sql, 0002_add_risk_flags.sql) to database/migrations for clearer schema history.

7. Code Review Findings and Recommendations

7.1 Summary of Findings

Overall, the repository is well-organized around the system's four-service architecture, with naming conventions and folder layout that make the codebase easy to navigate. Code quality is solid in the reviewed backend route and ML prediction function, following idiomatic patterns for each language, though a few controller functions and configuration values would benefit from refactoring. Version control practices are strong: a Gitflow-inspired branching model, conventional commit messages, and a demonstrated conflict-resolution merge show the team collaborates effectively through Git. Automated testing exists for the backend and ML service with passing results, but frontend test coverage and end-to-end integration testing

are comparatively thin. Documentation is largely complete but would benefit from a consolidated API reference and contributor guidelines.

7.2 Recommendations

Area	Recommendation
Code Quality	Extract long controller functions into smaller helpers; centralize configuration constants like <code>RISK_THRESHOLD</code> .
Repository Management	Adopt versioned migration filenames and add a <code>CONTRIBUTING.md</code> documenting the branching and review workflow.
Documentation	Publish an OpenAPI/Swagger reference and a non-Docker local setup guide.
Testing	Increase frontend unit test coverage and add at least one end-to-end test spanning all four services.
Future Enhancements	Introduce CI-enforced linting and test coverage thresholds on pull requests to develop, and add automated image-size checks for the ml-service Docker image.

Conclusion

This code review and repository evaluation confirms that the Smart Classroom Attendance, Engagement, and Performance Analytics System is implemented with a clear, service-oriented repository structure, reasonably clean and idiomatic code, disciplined version control practices, and largely complete documentation. The identified gaps — deeper frontend test coverage, a consolidated API reference, and finer-grained commit splitting in a few places — are incremental rather than structural, and addressing them would further strengthen the project's maintainability and readiness for collaborative, long-term development.